

Title

文件编号: CODE-CODE

版本: 0.0.0

更新历史记录

版本	Date	更新描述
----	------	------

目录

1 环境信息..... 6

1.1 开发.....6

1.2 测试.....6

2 公共信息..... 7

2.1 错误码.....7

2.2 请求头（Header）7

3 接口信息..... 8

3.1 整车状态.....8

3.1.1 接口功能说明8

3.1.2 请求参数8

3.1.3 响应参数8

3.2 点火状态查询列表..... 10

3.2.1 接口功能说明 10

3.2.2 请求参数 10

3.2.3 响应参数 11

3.3 充电状态列表..... 12

3.3.1 接口功能说明 12

3.3.2 请求参数 12

3.3.3 响应参数 12

4 签名机制说明文档.....15

4.1 1. 概述..... 15

4.2 2. 签名算法流程（面向接入方） 15

4.2.1	2.1 收集参数.....	15
4.2.2	2.2 参数值处理规则.....	16
4.2.3	2.3 签名计算步骤.....	16
4.3	3. 参数处理详细规则（面向内部开发）	16
4.3.1	3.1 请求参数转换 (convertRequestParams).....	16
4.3.2	3.2 路径变量提取.....	17
4.3.3	3.3 递归扁平化 (flattenParameters).....	17
4.3.4	3.4 值转换规则 (convertValueToString).....	17
4.3.5	3.5 JSON 序列化配置.....	17
4.3.6	3.6 HEX 编码规则	17
4.4	4. 完整计算示例	18
4.4.1	4.1 请求信息.....	18
4.4.2	4.2 参数收集.....	18
4.4.3	4.3 排序后的参数.....	18
4.4.4	4.4 拼接字符串	18
4.4.5	4.5 计算签名.....	19
4.5	5. 边界情况与注意事项	19
4.5.1	5.1 空值处理.....	19
4.5.2	5.2 JSON 判断逻辑	19
4.5.3	5.3 列表排序.....	19
4.5.4	5.4 Map/Object 序列化	19
4.5.5	5.5 字符编码.....	20
•	5.6 签名长度.....	20
4.6	6. 核心代码参考	20

4.6.1	6.1 签名验证入口.....	20
4.6.2	6.2 核心方法.....	20
4.6.3	6.3 相关类.....	20
4.7	7. 接入方快速接入指南	21
4.7.1	7.1 必需信息.....	21
4.7.2	7.2 签名生成伪代码.....	21
4.7.3	7.3 请求示例.....	22
4.7.4	7.4 Java 示例.....	22

1 环境信息

1.1 开发

- 地址: <https://test-data-open-platform.leapmotor.com>
- **appId:** 57fb1e5c037a4e2ca0d8e20a20114173
- **appSecret:** czjVYxu1RqcD4TdGYmDDP7rcqay0zTKxUfLrfE/YUayCv0nkeHwD79a4FD3zqmlG

1.2 测试

- 地址: <https://test-data-open-platform.leapmotor.com>
- **appId:** 57fb1e5c037a4e2ca0d8e20a20114173
- **appSecret:** czjVYxu1RqcD4TdGYmDDP7rcqay0zTKxUfLrfE/YUayCv0nkeHwD79a4FD3zqmlG

2 公共信息

2.1 错误码

- 其它未声明的，只要是非 0 都是错误，错误描述看 `message`

code	含义	说明
0	OK	请求处理成功
200400	Bad Request	请求头、请求参数、请求体校验不通过
200401	Unauthorized	验签失败
200403	Forbidden	权限不足
200429	Too Many Requests	请求过载触发流控
200500	Internal Server Error	服务内部处理请求报错

2.2 请求头（Header）

- 所有接口均需在请求头（Header）中携带以下参数

参数名	类型	必填	说明
X-App-Id	String	是	接入方唯一标识
X-Signature	String	是	SHA-256 + HEX 签名
X-Timestamp	Long	是	Unix 毫秒级时间戳，5 分钟内有效
X-Nonce	String	是	随机数

3 接口信息

3.1 整车状态

3.1.1 接口功能说明

- 查询整车状态数据，包含 ACC 模式、车速等信息
- 接口地址： /api/v1/cars/{vin}/vehicle-status
- 接口类型： GET

3.1.2 请求参数

字段	类型	必填	备注
vin	String	是	车辆 VIN 码

3.1.3 响应参数

字段	类型	备注
code	Integer	响应码，0 表示成功
message	String	提示信息
data	Object	整车状态数据

- data 元素展开信息

字段	类型	备注
collectionTime		服务器时间
mmiCollectTime		车机采集时间
doorState	arr[int]	门状态 [主驾驶门状态,副驾驶门状态,右后门状态,左后门状态,后备箱门状态,主驾驶窗状态,副驾驶窗状态,右后窗状态,左后窗状态,前盖状态] 0:off 1:on
airConditionSwitch		空调开关 0x00:关 0x01:开

字段	类型	备注
openDoorMethod		判断开门方式 00:none 01:use key 02:not use key 03:invalid
positionLights		位置灯 0x00:关 0x01:开
tirePressure	arr[int]	胎压 [左前轮胎,右前轮胎,左后轮胎, 右后轮胎]
soc		soc
batteryVoltage		电池电压
gearSignal		档位信号 00:P 01:R 02:N 03:D
abs_escIsSuedSpeed		ABS/ESC 发出车速
fireStatus		车辆上下电状态 00: fire on 01:fire off
keyDoorPosition		钥匙门位置 on2 电 0x00:off 0x01:on
gpsLocation	arr[double]	Gps 位置 [经度,纬度]
drivingMode		驾驶模式 00:standard mode 01:comfort mode 02:sport mode 03:economic mode
autoHoldSwitch		AutoHold 开关 0x00:disable 0x01:enable
doorLockState	arr[int]	门锁状态 [主驾驶门锁状态,副驾驶门锁状态,右后门锁状态,左后门锁状态,后备箱门锁状态] 0x00:off 0x01:on
temperature		车内温度
rearFogLights		后雾灯 0x00:关 0x01:开
escSwitch		ESC 开关 0x00:关 0x01:开
numberOfTirePressureFailures		故障个数 0-100000
chargingStatus		充电状态 0:未充电 1: 充电中 2: 充电结束 3: 充电故障 4: 充电预约中/等待中
batteryPower		蓄电池电量 soc

```
{
  "code": 0,
  "message": "success",
  "data": {
    "doorState": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    "collectionTime": "2026-06-13 13:02:27",
    "airConditionSwitch": 1,
    "openDoorMethod": 1,
    "positionLights": 0,
    "tirePressure": [255, 253, 255, 253],
```

```
"soc": 68,
"batteryVoltage": -1.0,
"gearSignal": 3,
"abs_escIsSuedSpeed": 65.475,
"fireStatus": 1,
"keyDoorPosition": 1,
"gpsLocation": [114.510666, 37.141575],
"drivingMode": 1,
"autoHoldSwitch": 1,
"doorLockState": [1, 1, 1, 1, 0],
"temperature": -1.0,
"rearFogLights": 0,
"escSwitch": 1,
"numberOfTirePressureFailures": 0,
"chargingStatus": 0,
"mmiCollectTime": "2026-06-13 13:02:27",
"batteryPower": 255
}
}
```

3.2 点火状态查询列表

3.2.1 接口功能说明

- 分页查询车辆点火/熄火状态历史记录，包含行程里程、能耗等信息
- 接口地址： /api/v1/cars/{car-id}/trace-points/fire-states
- 接口类型： GET

3.2.2 请求参数

字段	类型	必填	备注
beginTime	String	是	开始日期
endTime	String	是	结束日期
carId	Long	是	车辆内部 ID
pageNum	Integer	是	页码，从 1 开始
pageSize	Integer	是	分页大小，最大 100

3.2.3 响应参数

字段	类型	备注
code	Integer	响应码，0 表示成功
message	String	提示信息
data	Object	分页数据

- data 展开信息

字段	类型	备注
total	Integer	总记录数
pageNum	Integer	当前页码
pageSize	Integer	每页条数
list	Array	点火记录列表

- list 元素展开信息

字段	类型	备注
id	Long	记录 ID
fireOnTime	Date	点火时间
fireOffTime	Date	熄火时间
carId	Long	车辆编号
traceMileage	Double	本次行程行驶里程
originalSoc	Short	起始电量
endSoc	Short	结束电量
consumeOil	Integer	本次里程消耗油量
endLocation	String	结束位置(经纬度)
consumeEnergy	Float	本次行程消耗能量值

```
{
  "code": 0,
  "message": "success",
  "data": {
    "total": 50,
    "pageNum": 1,
    "pageSize": 20,
    "list": [
      {
        "id": "12345",
```

```
    "fireOnTime": "2024-01-15 08:30:00",
    "fireOffTime": "2024-01-15 09:15:00",
    "carId": 10001,
    "traceMileage": 25.00,
    "originalSoc": 85,
    "endSoc": 80,
    "consumeOil": 0,
    "endLocation": "120.15,30.28",
    "consumeEnergy": 3.5
  }
]
}
```

3.3 充电状态列表

3.3.1 接口功能说明

- 分页查询充电状态记录列表
- 接口地址: /api/v1/cars/{car-id}/charge-profiles/states
- 接口类型: GET

3.3.2 请求参数

字段	类型	必填	备注
carId	String	是	车辆 ID
beginTime	String	是	开始时间
endTime	String	是	结束时间
pageNum	Integer	是	页码
pageSize	Integer	是	每页大小

3.3.3 响应参数

字段	类型	备注
code	Integer	响应码, 0 表示成功

字段	类型	备注
message	String	提示信息
data	Object	分页数据

• data 展开信息

字段	类型	备注
total	Integer	总记录数
pageNum	Integer	当前页码
pageSize	Integer	每页条数
list	List	数据列表

• list 元素展开信息

字段	类型	备注
beginCharge	Date	开始充电时间
endCharge	Date	充电结束时间
beginChargePower	Integer	开始充电时电量
endChargePower	Integer	充电结束时电量
chargeStopReason	Integer	充电停止原因
chargeMode	String	充电模式
chargeResult	Integer	充电结束时状态
location	String	位置 经纬度
chargeInEC	Float	充入电量
packAddEC	Float	电池包增加电量
acEC	Float	空调能耗
compressorEC	Float	压缩机能耗
coldTime	Float	制冷时长
heatTime	Float	制热时长
acUseTime	Integer	AC 使用时长

```
{
  "code": 0,
  "message": "success",
  "total": 10,
  "pageNum": 1,
  "pageSize": 20,
```

```
"list": [  
  {  
    "beginCharge": "2024-01-15 10:00:00",  
    "endCharge": "2024-01-15 12:00:00",  
    "beginChargePower": 20,  
    "endChargePower": 95,  
    "chargeStopReason": 1,  
    "chargeMode": "xxx",  
    "chargeResult": 20,  
    "location": "xxx",  
    "chargeInEC": 80.1,  
    "packAddEC": 80.1,  
    "acEC": 80.1,  
    "compressorEC": 80.1,  
    "coldTime": 80.1,  
    "heatTime": 80.1,  
    "acUseTime": 900  
  }  
]  
}
```

4 签名机制说明文档

4.1 1. 概述

本系统使用基于 SHA-256 + HEX 编码的签名机制来验证请求的完整性和合法性。

签名 Header: X-Signature

核心流程:

- 1. 收集 Header 参数, X-Timestamp、X-Nonce
- 2. 收集所有请求参数 (Query 参数 + Path 变量)
- 3. 追加 appSecret 参数
- 4. 参数值扁平化为字符串
- 5. 按 key 字典序排序
- 6. 拼接为 key1=value1#key2=value2#... 格式
- 7. 对拼接字符串进行 SHA256 哈希后 HEX-DIGEST 编码
- 注意: X-Timestamp 和 X-Nonce 签名时拼接名称使用 timestamp 和 nonce

4.2 2. 签名算法流程 (面向接入方)

4.2.1 2.1 收集参数

参与签名计算的参数包括:

参数来源	说明
Header 参数	X-Timestamp(header 名) -> timestamp(拼接名), X-Nonce(header 名) -> nonce(拼接名)
Query 参数	URL 中 ? 后的所有参数, 如 ?name=foo&type=bar
Path 变量	URL 模板中的路径变量, 如 /users/{id} 中的 id
appSecret	分配给应用的密钥, 固定追加到参数中

4.2.2 2.2 参数值处理规则

参数值在参与签名前会被转换为字符串：

值类型	转换规则	示例
字符串	直接使用	"foo" → "foo"
数字	转为字符串	123 → "123"
布尔值	转为字符串	true → "true"
数组/列表	元素逗号分隔	["a", "b"] → "a,b"
对象/Map	转为 JSON（key 排序）	{"b":1, "a":2} → {"a":2, "b":1}
null	转为空字符串	null → ""

注意：如果参数值是 JSON 字符串（以 { 或 [开头），会被解析后再序列化为规范 JSON（key 排序）。

4.2.3 2.3 签名计算步骤

步骤 1：将所有参数值转为字符串（按 2.2 规则）
步骤 2：按参数 key 的字典序升序排序
步骤 3：拼接为 key1=value1#key2=value2#...#keyN=valueN
步骤 4：对拼接字符串计算 SHA-256 哈希（字节数组）
步骤 5：将字节数组转为小写 HEX 字符串（64 字符）
步骤 6：将编码结果放入请求头 X-Signature

4.3 3. 参数处理详细规则（面向内部开发）

4.3.1 3.1 请求参数转换 (convertRequestParams)

HttpServletRequest.getParameterMap() 返回 Map<String, String[]>，需要转换为 Map<String, Object>：

```
String[] values = rawParamMap.get(key);

if (values.length == 0):
    result[key] = ""

if (values.length == 1):
    val = values[0]
```



```
if isJsonString(val):    // 以 { 或 [ 开头
    result[key] = JSON.parse(val)    // 解析为 Map 或 List
else:
    result[key] = val

if (values.length > 1):
    result[key] = List<String>(values)    // 保持为字符串列表
```

4.3.2 3.2 路径变量提取

通过 Spring 的 `UriTemplate` 从请求 URI 中提取路径变量:

```
String lookupPath = (String) request.getAttribute(HandlerMapping.BEST_MATCHING_PATTERN_ATTRIBUTE);
UriTemplate uriTemplate = new UriTemplate(lookupPath);
Map<String, String> uriVars = uriTemplate.match(request.getRequestURI());
processedParams.putAll(uriVars);
```

4.3.3 3.3 递归扁平化 (`flattenParameters`)

将 `Map<String, Object>` 扁平化为 `Map<String, String>`:

```
for each (key, value) in params:
    result[key] = convertValueToString(value)
```

4.3.4 3.4 值转换规则 (`convertValueToString`)

<code>null</code>	→ <code>""</code>
<code>String</code>	→ 原值
<code>Number/Boolean</code>	→ <code>String.valueOf(value)</code>
<code>List</code>	→ 元素用 <code>","</code> 连接
<code>Array</code>	→ 转为 <code>List</code> 后同上
<code>Map/Object</code>	→ <code>ObjectMapper.writeValueAsString(value)</code> // key 排序的紧凑 JSON
其他	→ <code>String.valueOf(value)</code>

4.3.5 3.5 JSON 序列化配置

```
ObjectMapper.configure(SerializationFeature.ORDER_MAP_ENTRIES_BY_KEYS, true);    // key 排序
ObjectMapper.configure(SerializationFeature.INDENT_OUTPUT, false);                // 紧凑输出
```

4.3.6 3.6 HEX 编码规则

```
private static String bytesToHex(byte[] bytes) {
    StringBuilder sb = new StringBuilder(bytes.length * 2);
    for (byte b : bytes) {
```

```
        sb.append(String.format("%02x", b)); // 小写十六进制
    }
    return sb.toString();
}
```

- 每个字节转为 2 位小写十六进制字符
- SHA-256 输出 32 字节，最终 HEX 字符串为 64 字符

4.4 4. 完整计算示例

4.4.1 4.1 请求信息

```
URL: GET /api/v1/users/123?name=张三&tags=a&tags=b&info={"age":30,"city":"Shanghai"}
Header: X-App-Id: app001
appSecret: my_secret_key
```

4.4.2 4.2 参数收集

Key	原始值	转换后
name	"张三"	"张三"
tags	["a", "b"]	"a,b"
info	{"age":30,"city":"Shanghai"}	{"age":30,"city":"Shanghai"} (JSON 解析后重新序列化, key 已排序)
id (path)	"123"	"123"
appSecret	"my_secret_key"	"my_secret_key"

4.4.3 4.3 排序后的参数

```
appSecret=my_secret_key
id=123
info={"age":30,"city":"Shanghai"}
name=张三
tags=a,b
```

4.4.4 4.4 拼接字符串

```
appSecret=my_secret_key#id=123#info={"age":30,"city":"Shanghai"}#name=张三#tags=a,b
```

4.4.5 4.5 计算签名

```
SHA-256 哈希: sha256("appSecret=my_secret_key#id=123#info={\"age\":30,\"city\": \"Shanghai\"}#name=张三#tags=a,b")
HEX 编码: <64 位小写十六进制字符串>
```

最终 `X-Signature` 的值为上述 HEX 编码结果（64 字符）。

4.5 5. 边界情况与注意事项

4.5.1 5.1 空值处理

- 空值（`null` 或空字符串）参与签名计算，不会被跳过
- 空列表转为空字符串 `""`

4.5.2 5.2 JSON 判断逻辑

```
private static boolean isJsonString(String str) {
    str = str.trim();
    return (str.startsWith("{") && str.endsWith("}"))
        || (str.startsWith("[") && str.endsWith("]"));
}
```

- 仅判断首尾字符，不验证 JSON 合法性
- 解析失败时回退为普通字符串处理

4.5.3 5.3 列表排序

- 当前实现中列表元素**不排序**，保持原始顺序
- 如需确定性签名，调用方应保证列表参数顺序一致

4.5.4 5.4 Map/Object 序列化

- 使用 Jackson 序列化，**key 自动排序**
- 输出为紧凑格式（无空格、无换行）

4.5.5 5.5 字符编码

- SHA-256 计算时使用指定字符集（通常 UTF-8）将字符串转为字节数组
- 确保客户端和服务端使用相同字符集

- ## 5.6 签名长度

- SHA-256 输出 32 字节
- HEX 编码后固定为 **64 字符**（小写）

4.6 6. 核心代码参考

4.6.1 6.1 签名验证入口

```
// Signature.java
public static boolean verifyRequestSign(HttpServletRequest request, String appSecret, Charset charset)
```

4.6.2 6.2 核心方法

方法	作用
verifyRequestSign	从 HTTP 请求中提取参数并验证签名
convertRequestParams	将 <code>String[]</code> 参数转换为 <code>Object</code>
flattenParameters	递归扁平化参数为 <code>Map<String, String></code>
convertValueToString	将任意值转换为签名字符串
sha256StringAndHex	计算 SHA-256 并转为 HEX 字符串
sha256String	计算 SHA-256 哈希字节数组
bytesToHex	字节数组转小写 HEX 字符串

4.6.3 6.3 相关类

- `com.leapmotor.platform.util.Signature` — 签名工具类
- `com.leapmotor.platform.util.Headers` — HTTP Header 常量定义
- `SignatureValidationInterceptor` — 签名验证拦截器

4.7 7. 接入方快速接入指南

4.7.1 7.1 必需信息

- `appId`: 应用标识
- `appSecret`: 应用密钥（由平台分配，不传输）

4.7.2 7.2 签名生成伪代码

```
import hashlib
import json

def generate_signature(params: dict, app_secret: str) -> str:
    # 1. 追加 appSecret
    params["appSecret"] = app_secret

    # 2. 转换值为字符串
    flat = {}
    for k, v in params.items():
        if isinstance(v, (dict, list)):
            # JSON 序列化, key 排序
            flat[k] = json.dumps(v, sort_keys=True, ensure_ascii=False, separators=(',', ':'))
        elif isinstance(v, list):
            flat[k] = ",".join(str(x) for x in v)
        elif v is None:
            flat[k] = ""
        else:
            flat[k] = str(v)

    # 3. 按 key 排序
    sorted_keys = sorted(flat.keys())

    # 4. 拼接
    content = "#".join(f"{k}={flat[k]}" for k in sorted_keys)

    # 5. SHA-256 + HEX
    sha256_hash = hashlib.sha256(content.encode('utf-8')).hexdigest()

    return sha256_hash # 64 位小写十六进制字符串
```

4.7.3 7.3 请求示例

```
GET /api/v1/users/123?name=test HTTP/1.1
Host: api.example.com
X-App-Id: app001
X-Signature: <64 位小写 HEX 字符串>
X-Timestamp: Unix 毫秒级时间戳, 5 分钟内有效
X-Nonce: 随机数
```

4.7.4 7.4 Java 示例

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.*;

public class SignatureGenerator {

    public static String generateSignature(Map<String, Object> params, String appSecret) throws
Exception {
        // 1. 追加 appSecret
        params.put("appSecret", appSecret);

        // 2. 转换值为字符串
        Map<String, String> flat = new TreeMap<>();
        for (Map.Entry<String, Object> entry : params.entrySet()) {
            flat.put(entry.getKey(), convertValueToString(entry.getValue()));
        }

        // 3. 拼接
        StringBuilder sb = new StringBuilder();
        for (Map.Entry<String, String> entry : flat.entrySet()) {
            if (sb.length() > 0) sb.append("#");
            sb.append(entry.getKey()).append("=").append(entry.getValue());
        }

        // 4. SHA-256 + HEX
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
        return bytesToHex(hash);
    }

    private static String convertValueToString(Object value) {
        if (value == null) return "";
        if (value instanceof Map || value instanceof List) {
            // 使用 Jackson 序列化为 JSON
            return new ObjectMapper().writeValueAsString(value);
        }
        return String.valueOf(value);
    }
}
```

```
private static String bytesToHex(byte[] bytes) {  
    StringBuilder sb = new StringBuilder(bytes.length * 2);  
    for (byte b : bytes) {  
        sb.append(String.format("%02x", b));  
    }  
    return sb.toString();  
}  
}
```